Lightweight Ontology Architecture

Architecture, business process understanding and evidence of agent execution. Extend the existing services and keep diagrams and verification artifacts inside the repository.

1. Connected sources
Existing database connectors; optional OpenMetadata; business documents and glossary

↓

2. Metadata filtering
Select relevant schemas, tables, columns, descriptions and business terms

↓

3. LLM ontology builder
Extract concepts and activities > Normalize names > Classify layers > Propose relationships

↓

4. Validation and SME review
Existing JSON Schema / optional Pydantic; RDFLib; pySHACL; accept, correct or reject

↓

5. Versioned ontology store
Existing database; RDF/OWL serialization; source mappings, evidence and process definitions

↓

6. Fast ontology retrieval
Exact index > Metadata-filtered search > Relationship traversal > Cache

↓

7. Existing agent orchestrator
Retrieve definitions > Select authorized tools > Ground fields and queries > Execute

↓

8. Internal evidence
Ontology graph; process diagram; versioned run trace; verified query and result

Corrections return to the builder before publication. Source changes trigger revalidation of affected definitions. Retrieval and execution continue to enforce existing access and tool permissions.

How the whole architecture flows

Build and publish the knowledge

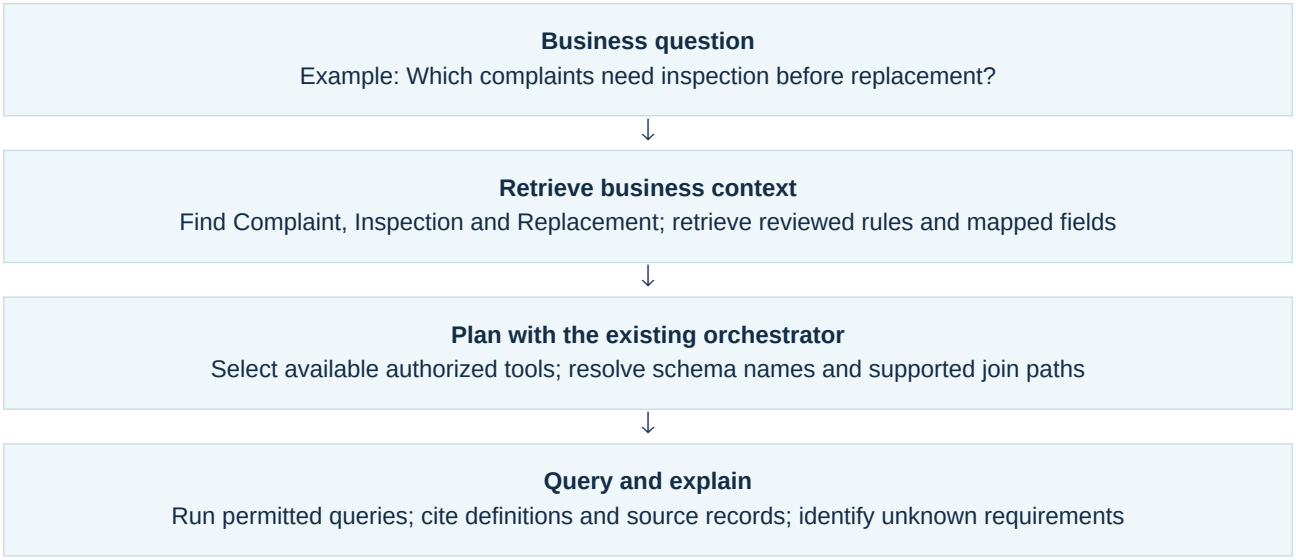
Connect and discover. Existing connectors read bounded source metadata. Add selected policies, SOPs, application descriptions and glossary definitions as evidence; OpenMetadata can supply catalogue context if integrated.

Filter and build. Select relevant source areas. The LLM proposes business objects, activities, roles, rules and relationships. Normalize synonymous terms while preserving source-specific mappings. Assign layers and distinguish definitions from individual records.

Validate and approve. Structural checks verify output shapes and references. RDFLib represents the graph; SHACL checks supported constraints. An SME verifies business meaning, sequence and policy authority. Missing or conflicting evidence remains an explicit open question.

Publish a version. Save approved definitions, relationships, evidence, source fingerprints and mappings in the existing database. Export RDF/OWL. Keep drafts separate, and retain immutable version history.

Use the knowledge for a business request



Ontology supplies meaning and context. Database constraints and existing query grants determine executable joins. Operational approvals and write permissions remain with existing application controls.

Refresh safely

When source metadata or a policy changes, identify affected mappings and rules, mark them for review, and publish a revised version. Trace each plan to the version it used; invalidate or refresh cached context by version and source fingerprint.

How business processes are represented

The ontology describes what the business works with and how activities relate. A process definition captures the reviewed order of activities, decision conditions and handoffs. Individual cases follow that definition using their actual events and records.

Illustrative complaint-to-resolution process

This is an example to validate with business owners, not an assertion about the organization's current process.

Complaint received
Customer or service team creates a complaint linked to an engine or product

↓

Triage and evidence collection
Capture symptoms, identity, dates and supporting records; flag missing information

↓

Policy assessment and investigation
Evaluate applicable approved policy; arrange inspection when the reviewed rule requires it

↓

Business decision
Authorized people decide repair, replacement, rejection or further investigation

↓

Fulfillment and closure
Record the permitted action, outcome and closure; monitor recurrence when relevant

What the process model records

Objects: Complaint, Product, Inspection, Decision and Replacement.
Activities and roles: who receives, investigates, decides and fulfills.
Relationships: complaint concerns product; inspection investigates complaint; decision is supported by evidence.
Process rules: preconditions, branches, required evidence and approval roles.
Source mappings: which datasets, fields and event types supply each fact.

Keep process order distinct from class inheritance. A relationship such as "replacement follows approval" describes a dependency; it does not make Replacement a subclass of Approval. The agent uses these definitions to interpret requests, while the existing workflow engine controls execution.

How processes are inferred and proven

Infer from several types of evidence

1. Gather evidence. Schemas reveal objects and structural links. SOPs and policies describe intended steps and authority. Application logic describes implemented behavior. Timestamped events or audit logs, if available and authorized, show observed sequences.

2. Extract candidates. The LLM proposes activities, actors, inputs, outputs, statuses and rules. Attach evidence references to each candidate and distinguish observed facts, model interpretations and hypotheses.

3. Assemble a candidate process graph. Connect activities using explicit documented dependencies, supported code paths and available event correlations. Infer branches only where conditions are supported. Tables and foreign keys alone cannot establish an entire business process.

4. Check uncertainty and contradictions. Compare documented, implemented and observed behavior. Repeated event order is a pattern, not proof of a mandatory rule or causality. Record missing steps, ambiguous handoffs and unsupported approvals rather than filling them in.

5. Review and publish. An SME confirms the proposed sequence, exceptions and authority. Publish the reviewed process alongside its ontology version. New evidence revises the draft; it never silently creates business policy.

Show that the ontology influenced the agent

Use a business term that differs from database naming. Record the retrieved definition and exact source mapping, the plan step that used it, the selected authorized tool, generated query and actual result. Check the query against an independently expected result. A trace with unused retrieved definitions is insufficient evidence of grounding.

Internal artifacts: no additional site

Architecture diagram: Markdown/Mermaid for the component flow.

Ontology and process graphs: SVG exports showing concepts, mapped fields, activity order, branches and evidence status.

Per-run trace: Markdown/JSON: request, ontology version, retrieved IDs, step bindings, tools, query, result and citations.

Verification report: Structural/SHACL outcomes, SME decisions, unresolved questions and expected-versus-actual query checks.

Simple implementation sequence

Reuse discovery and filtering; generalize the current ontology builder; add reviewed process definitions and lightweight approval; extend existing retrieval and agent bindings; demonstrate one dataset and a few questions before expanding.

First deliverable: one working vertical slice with an ontology graph, a reviewed process diagram and an inspectable agent trace. Existing components are reused; broader process inference and approval are proposed extensions, not claims of live implementation.